

AsA

Master Build Bible v1.0

TTT-first

AsA - AI Trading Intelligence Terminal

Master product, architecture, data, API, UX, research and build specification

This document is a reconstruction and normalization of the existing AsA codebase, its real runtime contracts, the earlier research project, the initial TTT-based project, and the final product requirements.

Primary purpose: give an external agent (including Fable 5.1) enough precise context to build the frontend without inventing data, sources, capabilities, or behavior.

Build principle: build what is real, expose what is unavailable, and never replace missing evidence with decorative UI.

Prepared from workspace inspection and current TTT web evidence. Technical terms intentionally remain in English where they map directly to code/API concepts.

0. Executive Summary

One-line definition: AsA is an AI-native crypto-futures trading intelligence terminal that watches TTT market data, interprets multi-timeframe price structure and market context, applies a user-defined strategy and psychology framework, calculates risk, produces an annotated chart and advisory signal, and stops before execution.

The non-negotiable source-of-truth rule

The final product is **TTT-only for venue truth**. The current rebuilt codebase contains a public Binance adapter/failover path, but the desired product specification here overrides that for the final product: no other venue may replace TTT price/candles/venue facts in the terminal. If TTT does not expose a metric, the correct UI state is **UNAVAILABLE**, not substitution.

What already exists in the current codebase

Backend

FastAPI + uvicorn, TTT REST adapter, data engine, SQLite/WAL, chart analysis, structure, regime, psychology, AI provider router, risk engine, opportunity engine, signal/outbox, backtest/validation, research API, WebSocket.

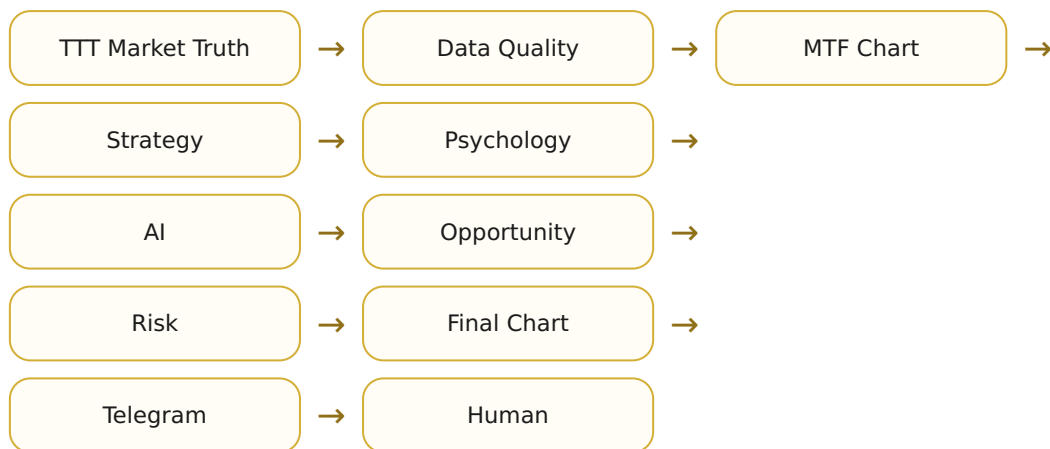
Frontend

Vanilla ES-module SPA, Lightweight Charts vendor, multiple pages, live WS client, dashboard/market/chart/signals/AI/research/settings/logs/system pages.

What is still fundamentally unfinished

- **TTT-only operational purity:** remove/disable cross-venue market truth in the final product and make provenance explicit in every visible value.
- **True live UX:** the browser must show current TTT data, actual freshness, actual candle updates, and real connection states.
- **Chart-first experience:** chart is the center of the product, not a decorative card.
- **User strategy integration:** the personal strategy is not fully encoded as a source-defined StrategySpec in the current snapshot; never invent its missing rules.
- **Local AI:** move from provider-only architecture toward an optional local AI service later, with the same structured contract.

1. Product North Star and Operating Philosophy



Core doctrine

- **No fake success.** Every number is measured, derived by a documented formula, or explicitly labeled unavailable/proxy.
- **TTT is the venue truth.** Venue-specific prices, candles, contract metadata and execution-context facts belong to TTT.
- **AI is bounded.** It reasons over structured market context and chart evidence; it cannot bypass risk and cannot execute.
- **Human remains the executor.** AsA never places orders, cancels orders, transfers funds, deposits or withdraws.
- **Research is separate from proof.** A backtest is evidence for a hypothesis, not a guarantee.
- **UI is observable.** If a worker is stale, the UI says stale; if a metric is unavailable, the UI says unavailable.

User experience target

The product should feel like an AI command center for market structure, not a generic signal website and not an administrative dashboard. The central interaction is: **select market → inspect live chart → inspect AI evidence → inspect risk → receive/ignore signal.**

2. Current Repository - Exact Inventory

The inspected rebuilt workspace contains **52 Python files** under `backend/asa/`, 16 frontend/vendor files, 30 test/runtime files in the tests tree, and 10 documentation files.

Top-level structure

```
asa/
├── backend/
│   ├── main.py
│   └── asa/
│       ├── agents/
│       ├── ai/
│       ├── api/
│       ├── backtest/
│       ├── chart/
│       ├── data/
│       ├── opportunity/
│       ├── psychology/
│       ├── risk/
│       ├── signal/
│       ├── strategy/
│       ├── bus.py
│       ├── config.py
│       ├── db.py
│       ├── exit_policy.py
│       └── models.py
├── frontend/
├── tests/
├── docs/
├── data/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── pyproject.toml
```

Backend module map

File	Lines	Main classes	Top-level functions
backend/asa/ __init__.py	2	-	-
backend/asa/agents/ __init__.py	1	-	-
backend/asa/agents/ orchestrator.py	451	Orchestrator	-
backend/asa/ai/ __init__.py	2	-	-
backend/asa/ai/ confidence.py	189	-	_regime_score, _psych_score, _data_score, _mtf_score, compute_confidence
	641		_bundle_price_range, _finite, _bundle_ts_range,

File	Lines	Main classes	Top-level functions
backend/asa/ai/providers.py		LLMProvider, OpenAI, Anthropic, Gemini, XAI, HeuristicAnalyst	validate_annotations, sanitize_annotations, _heuristic_verdict, _parse_json, _clip, build_providers
backend/asa/ai/router.py	107	AIRouter	-
backend/asa/api/__init__.py	1	-	-
backend/asa/api/deps.py	143	RateLimiter	create_app, _frontend_dir, _f
backend/asa/api/routes_analysis.py	323	AnalyzeBody	analysis, _mtf_context, _build_bundle, _latest_ai_annotations, _safe, ai_analyze, _candles_from_bundle
backend/asa/api/routes_market.py	258	-	symbols, prices, candles, trades, derivatives, deriv_features, board, coverage
backend/asa/api/routes_research.py	166	BacktestBody	hypotheses, strategies, ai_calls, backtest, _run_job, backtest_status
backend/asa/api/routes_signals.py	124	-	_hydrate, opportunities, signals, journal_list, journal_upsert, journal_delete, signal_one, chart
backend/asa/api/routes_system.py	157	RiskOverride, AIPolicyOverride	health, status, events, config, apply_persisted_risk, config_risk_get, config_risk, logs, config_ai_get, config_ai, apply_persisted_ai
backend/asa/api/ws.py	70	-	ws_endpoint
backend/asa/backtest/__init__.py	2	-	-
backend/asa/backtest/engine.py	179	BtConfig, _Pos	_fee, run_backtest, _np_atr
backend/asa/backtest/metrics.py	101	-	compute_metrics, _std, _annualized
backend/asa/backtest/validation.py	259	-	_slim, oos_split, walk_forward, monte_carlo, sensitivity, cost_robustness, regime_breakdown, bootstrap_ci, deflated_performance, _norm_pp1, multiple_testing_note
backend/asa/bus.py	55	Bus	-
	3	-	-

File	Lines	Main classes	Top-level functions
backend/asa/chart/____init__.py			
backend/asa/chart/annotate.py	369	ChartRenderer	-
backend/asa/chart/indicators.py	168	-	_sma, _ema, rsi, atr, adx, vol_zscore, compute_indicators
backend/asa/chart/regime.py	180	-	_slope_atr, htf_trend, _atr_pct_median, _vol_modifier, _slope_flip, classify_regime
backend/asa/chart/structure.py	444	-	find_swings, classify_swings, detect_events, find_fvgs, _mark_fill, find_order_blocks, build_levels, find_liquidity, fib_levels, find_trendlines, analyze_structure
backend/asa/config.py	197	Settings	_env_secret, get_settings
backend/asa/data/____init__.py	1	-	-
backend/asa/data/base.py	74	FeedError, SourceUnavailable, FeedHealth, Source	-
backend/asa/data/binance.py	168	BinanceFeed	parse_binance_symbol, _tf_name
backend/asa/data/derivatives.py	387	Observation, TTTProvider, BinanceProvider, _Unavailable	capabilities, fetch, fetch_dict
backend/asa/data/engine.py	630	DataEngine	-
backend/asa/data/meta.py	78	-	_f, normalize_market_meta, tier_for_notional
backend/asa/data/normalize.py	103	-	sort_candles, dedupe_candles, validate_candles, detect_gaps, resample_candles, _tf_name
backend/asa/data/ttt.py	177	TTTFeed	parse_udf, _tf_name
backend/asa/db.py	884	Database	-
backend/asa/exit_policy.py	201	Fill, ExitState	status_label, remaining_frac, _fill_r, step_exit
backend/asa/logging_conf.py	57	_RingHandler	setup_logging, get_recent, _lvl_num
backend/asa/models.py	307	Candle, PricePoint, SwingPoint, StructureEvent, Trendline, Zone, Level,	-

File	Lines	Main classes	Top-level functions
		LiquidityPoint, StructureSummary, Regime, PsychologyMetric, PsychologyState, StrategySignal, AICall, ConfidenceBreakdown, RiskCheck, RiskDecision, Opportunity	
backend/asa/opportunity/__init__.py	1	-	-
backend/asa/opportunity/engine.py	499	OpportunityEngine	_now_id, _atr_arr, _opp_to_db, _last
backend/asa/psychology/__init__.py	1	-	-
backend/asa/psychology/engine.py	306	PsychologyEngine	_last, _feats, _last_taker_ratio, _last_taker_bs, _last_oi, _regime_label, _contradictions
backend/asa/risk/__init__.py	1	-	-
backend/asa/risk/engine.py	316	RiskEngine	round_to_tick, floor_to_step, _corr, _market_max_lev
backend/asa/signal/__init__.py	1	-	-
backend/asa/signal/dispatchers.py	33	DryRunDispatcher, NullDispatcher	default_dispatchers
backend/asa/signal/engine.py	250	SignalEngine	_backoff, _maybe_dict, build_message
backend/asa/signal/telegram.py	62	TelegramDispatcher	-
backend/asa/strategy/__init__.py	2	-	-
backend/asa/strategy/base.py	61	StrategyContext, Strategy	-
backend/asa/strategy/registry.py	37	StrategyRegistry	-
backend/asa/strategy/structure_break.py	171	ReferenceBOSRetrace, _FibZone	_last_valid, _tf_sec

3. Runtime configuration and canonical market universe

Canonical timeframes

1m, 5m, 15m, 30m, 45m, 1h, 2h, 4h, 8h, 1d.

Canonical 48-symbol universe

#	Symbol
1	1000PEPEUSDT
2	1000SHIBUSDT
3	AAVEUSDT
4	ADAUSDT
5	ALGOUSDT
6	APEUSDT
7	APTUSDT
8	ARBUSDT
9	ASTERUSDT
10	ATOMUSDT
11	AVAXUSDT
12	BANDUSDT
13	BCHUSDT
14	BNBUSDT
15	BTCUSDT
16	CAKEUSDT
17	DASHUSDT
18	DOGEUSDT
19	DOTUSDT
20	ENAUUSD
21	ETCUSDT
22	ETHUSDT

#	Symbol
23	FETUSDT
24	FILUSDT
25	HBARUSDT
26	HYPEUSDT
27	ICPUSDT
28	INJUSDT
29	KSMUSDT
30	LINKUSDT
31	LTCUSDT
32	NEARUSDT
33	NOTUSDT
34	ONDOUSDT
35	OPUSDT
36	PAXGUSDT
37	QNTUSDT
38	SANDUSDT
39	SOLUSDT
40	SUIUSDT
41	TRUMPUSDT
42	TRXUSDT
43	UNIUSDT
44	VETUSDT
45	WLDUSDT
46	XL MUSDT
47	XRPUSDT
48	ZECUSDT

Hard exclusion: TONUSDT is intentionally absent and blocked by design. The final UI should not silently reintroduce it through another API.

Current configuration realities

Setting	Current code reality	Final product expectation
Primary source	TTT	TTT only for venue truth
Failover source	Binance public adapter exists	Disable/remove as market-truth source in final product
Live core TFs	15m, 1h, 4h configured	4h macro + 1h context + 15m trigger
Scan	event-driven plus periodic safety sweep	keep bounded, observable, event-first
AI	provider chain + heuristic floor	local-first optional later; never fake provider status
Telegram	dry-run until token/chat configured	show exact state; send final chart only when truly configured
Execution	none	remain none

4. TTT Data Contract - What is actually verified

Verified public TTT endpoints in the current codebase

Endpoint	Purpose	Notes
GET /futures/markets	market metadata	all instruments + trading metadata
GET /futures/udf/history	OHLCV	TradingView UDF payload; resolution numeric; from/to epoch seconds
GET /futures/markets/trades	public trade tape	server caps at 50 records/call

Observed upstream constraints

- No verified WebSocket was found in the current source/docs; the adapter therefore uses REST polling rather than inventing a WS subscription format.
- History responses have a cap (current adapter stays under the cap using chunked requests).
- The forming candle can be duplicated; parser deduplicates.
- TTT currently has no native 1d UDF resolution in the code; the adapter synthesizes 1d from 8h and labels it.
- The candle endpoint was observed to lag by roughly one closed bar in the project validation notes; freshness must therefore be explicit.
- TTT can throttle aggressively under bursts; the data engine contains retries/circuit breaking/repair behavior, but final UX must make degraded freshness visible.

Current TTT adapter behavior

```
"""The True Trade REST adapter (primary source).

Verified against the live API (2026-09-03):
- GET /futures/markets          → all instruments + trading metadata
- GET /futures/udf/history      → TradingView UDF candles
  ?symbol=X&resolution=NN&from=&to= (from/to epoch SECONDS)
  response: {"s":"ok","t":[sec],"o":[str],"h":...,"l":...,"c":...,"v":...}
- GET /futures/markets/trades   → public tape (server caps at 50/call;
  the limit parameter is ignored beyond 50)

Known upstream properties (documented, worked around – not invented):
- NO WebSocket exists → live updates are REST polling.
- 5000-candle cap per response → chunked requests.
- The forming candle is duplicated at the end of responses → dedupe.
- No 1d resolution → synthesized from 8h (labeled).
- The candle endpoint lags ~1 closed bar (observed).
"""

from __future__ import annotations

import asyncio
import time
```

```

import httpx

from ..models import Candle
from .base import FeedError, Source, SourceUnavailable
from .normalize import dedupe_candles, resample_candles, sort_candles, validate_candles

# tf seconds → TTT resolution (string). 1d is NOT resolvable upstream.
RESOLUTIONS: dict[int, str] = {
    60: "1", 300: "5", 900: "15", 1800: "30", 2700: "45",
    3600: "60", 7200: "120", 14400: "240", 28800: "480",
}

CHUNK_CANDLES = 4500 # stay under the 5000 upstream cap

def parse_udf(d: dict, symbol: str, tf: str, tf_sec: int,
              now: int | None = None) -> list[Candle]:
    """Parse a TradingView UDF payload into validated, deduped candles."""
    if d.get("s") != "ok" or d.get("errors"):
        msg = str(d.get("errors") or d.get("s"))
        raise FeedError(f"TTT udf error: {msg}")
    now = now or int(time.time())
    rows: list[Candle] = []
    ts_list = d.get("t") or []
    for i, ts in enumerate(ts_list):
        ts = int(ts)
        try:
            o, h, l, c = (float(d["o"][i]), float(d["h"][i]), float(d["l"][i]),
                           float(d["c"][i]))
            v = float(d["v"][i]) if d.get("v") else 0.0
        except (IndexError, TypeError, ValueError) as e:
            raise FeedError(f"TTT udf row malformed at {ts}: {e}")
        closed = (ts + tf_sec) <= now
        rows.append(Candle(symbol=symbol, tf=tf, ts=ts, o=o, h=h, l=l, c=c,
                           v=v, closed=closed, source="ttt"))
    valid, _issues = validate_candles(rows)
    valid = dedupe_candles(valid) # forming candle is duplicated upstream
    return sort_candles(valid)

class TTTFee(Source):
    name = "ttt"

    def __init__(self, base_url: str = "https://apiv2.thetrue.trade.io",
                 timeout: float = 15.0, min_interval: float = 0.12,
                 max_concurrent: int = 6) -> None:
        super().__init__()
        self.base = base_url.rstrip("/")
        self.timeout = timeout
        self.min_interval = min_interval
        self._last_req = 0.0
        self._sem = asyncio.Semaphore(max_concurrent)
        self._client: httpx.AsyncClient | None = None

    async def _http(self) -> httpx.AsyncClient:
        if self._client is None or self._client.is_closed:
            self._client = httpx.AsyncClient(timeout=self.timeout)
        return self._client

    async def _throttle(self) -> None:
        wait = self.min_interval - (time.monotonic() - self._last_req)
        if wait > 0:
            await asyncio.sleep(wait)
        self._last_req = time.monotonic()

    async def _get(self, path: str, params: dict) -> dict | list:
        c = await self._http()
        async with self._sem:
            await self._throttle()
            try:
                r = await c.get(self.base + path, params=params)

```

```

        except httpx.HTTPError as e:
            self.health.mark_error(str(e))
            raise SourceUnavailable(f"TTT network error: {e}") from e
    if r.status_code in (451, 403):
        self.health.mark_error(f"TTT blocked: {r.status_code}")
        raise SourceUnavailable(f"TTT geo-blocked ({r.status_code})")
    if r.status_code != 200:
        self.health.mark_error(f"TTT HTTP {r.status_code} {path}")
        raise FeedError(f"TTT HTTP {r.status_code} for {path}: {r.text[:200]}")
    self.health.mark_ok()
    return r.json()

async def aclose(self) -> None:
    if self._client and not self._client.is_closed:
        await self._client.aclose()

# ----- markets
async def markets(self) -> list[dict]:
    data = await self._get("/futures/markets", {})
    if not isinstance(data, list):
        raise FeedError("TTT markets: unexpected payload")
    return data

# ----- candles
async def candles(self, symbol: str, tf_sec: int, start: int,
                  end: int) -> list[Candle]:
    if tf_sec == 86400:
        # 1d synthesized from 8h (labeled source)
        raw = await self._fetch_range(symbol, 28800, start, end)
        return resample_candles(raw, 86400)
    res = RESOLUTIONS.get(tf_sec)
    if res is None:
        raise SourceUnavailable(f"TTT has no resolution for {tf_sec}s")
    return await self._fetch_range(symbol, tf_sec, start, end, res)

async def _fetch_range(self, symbol: str, tf_sec: int, start: int,
                      end: int, res: str | None = None) -> list[Candle]:
    out: list[Candle] = []
    cur = start
    while cur < end:
        chunk_end = min(end, cur + CHUNK_CANDLES * tf_sec)
        data = await self._get("/futures/udf/history", {
            "symbol": symbol,
            "resolution": res or RESOLUTIONS[tf_sec],
            "from": cur,
            "to": chunk_end,
        })
        rows = parse_udf(data, symbol, _tf_name(tf_sec), tf_sec)
        rows = [r for r in rows if r.ts >= start and r.ts < end]
        if rows:
            out.extend(rows)
            cur = rows[-1].ts + tf_sec
        else:
            # no data in window – advance (gap or out-of-history)
            cur = chunk_end
        if cur >= end:
            break
    return out

# ----- trades
async def trades(self, symbol: str, limit: int = 50) -> list[dict]:
    data = await self._get("/futures/markets/trades", {"symbol": symbol,
                                                         "limit": min(limit, 50)})
    out = []
    for t in (data.get("trades") or [])[:limit]:
        try:
            out.append({
                "ts_ms": int(t["timestamp"]),
                "price": float(t["price"]),
                "size": float(t["size"]),
                "side": str(t.get("side", "")),
            })

```

```
        except (KeyError, TypeError, ValueError):
            continue
    return out

    async def price(self, symbol: str) -> float | None:
        t = await self.trades(symbol, limit=1)
        return t[0]["price"] if t else None

def _tf_name(sec: int) -> str:
    from .normalize import _TF_BY_SEC
    return _TF_BY_SEC.get(sec, f"{sec}s")
```

5. TTT Venue Truth versus Context Metrics

Important correction for future builds: the current repository contains a Binance public-futures adapter. The final AsA product requested here must not use Binance to replace TTT price/candle/contract truth. The frontend must make this distinction visually impossible to miss.

Rule for all venue-specific facts

Data	Final source	Fallback policy
price	TTT	Do not substitute another venue; show unavailable and log provenance
candles	TTT	Do not substitute another venue; show unavailable and log provenance
trades	TTT	Do not substitute another venue; show unavailable and log provenance
market metadata	TTT	Do not substitute another venue; show unavailable and log provenance
tick/step	TTT	Do not substitute another venue; show unavailable and log provenance
fees	TTT	Do not substitute another venue; show unavailable and log provenance
leverage/contract limits	TTT	Do not substitute another venue; show unavailable and log provenance
mark/index/funding/OI/orderbook/liquidations if and only if TTT public data actually exposes them	TTT	Do not substitute another venue; show unavailable and log provenance

Current official TTT web evidence

The current TTT web pages expose a futures trading page with mark price, index price, funding, open interest, chart, order book and recent trades in the product UI; current help-center material also documents perpetual mechanics, funding and risk-limit concepts. This is evidence that these concepts exist in the product, but it does **not** by itself prove a public endpoint. The implementation rule is therefore: **discover/verify the public API or data channel; otherwise show unavailable.**

External references: TTT home/product, futures contract pages, funding-rate page and perpetual mechanics help-center article. See source list at the end.

6. Market Data Engine - Current implementation and target behavior

Current components

Module	Current role
data/base.py	Source interface, feed health, SourceUnavailable/FeedError
data/ttt.py	TTT primary REST adapter, markets/history/trades
data/binance.py	public Binance adapter; must not be used as final venue truth
data/normalize.py	sort, dedupe, validate, gap detection, resampling
data/meta.py	market metadata normalization and leverage tier lookup
data/engine.py	polling/backfill/repair/live loops, freshness, source orchestration
data/derivatives.py	provider/capability and derivative observation seam

Target live behavior

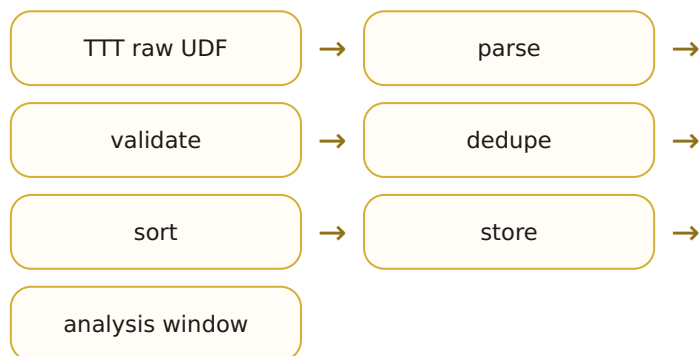
- TTT price sample must carry symbol, price, source=ttt, source_ts/received_ts and age.
- Closed candle events must be derived from TTT candles only and must be visibly timestamped.
- Per-symbol/per-timeframe freshness must be measured, not inferred from a global loop time.
- The system should prioritize 15m close → analysis; 1h close → context refresh; 4h close → macro refresh.
- Backfill must converge with bounded concurrency and remain honest under TTT throttling.
- A stale or missing candle set must produce INSUFFICIENT DATA, never an empty chart with pretend structure.

Latency target

The user wants sub-100ms ping. That must be treated as a **measured network/runtime target**, not a promise. The product should display RTT and data age. If TTT RTT is 180ms from a given connection, the UI must show 180ms.

7. Candle, normalization and data-quality semantics

Normalization chain



Quality flags

Flag	Meaning	UI response
fresh	recent and valid	normal
stale	age beyond threshold	amber + show age
gap	missing candle(s)	warning + coverage state
duplicate	repeated timestamp	hide duplicate + count
malformed	bad OHLC relationship or parse	exclude row + error count
derived	resampled from another timeframe	badge "DERIVED"
unavailable	provider did not provide metric	show "Unavailable"

Current code stores raw candle data in SQLite and computes indicators/structure on demand. This is a good separation: raw market truth should remain raw; derived analysis should remain reproducible from the stored input.

8. Chart Intelligence - What exists today

Indicators

Wilder RSI, ATR, ADX, EMA and volume z-score are implemented in `chart/indicators.py`.

Structure engine

- Fractal swings (k=2)
- HH/HL/LH/LL classification
- close-break BOS/CHoCH state machine
- support/resistance clustering
- 3-candle FVG
- order blocks with breaker tracking
- Fibonacci levels
- liquidity points
- trendlines
- range position

Regime engine

The current regime classifier uses ADX/EMA slope/ATR% plus a higher-timeframe gate. It exposes evidence and conflicts.

Chart rendering

`chart/annotate.py` renders matplotlib PNGs for signals; the frontend uses Lightweight Charts for the interactive browser chart.

Design rule for Fable: the browser chart must be driven by the same backend analysis objects used by the signal pipeline. Do not recreate trading logic in JavaScript.

9. Strategy Engine - Current reference strategy and what is missing

Current strategy contract

`StrategyContext` carries symbol, timeframe, candles, indicators, structure, regime, psychology, HTF context, price and parameters; the strategy returns an advisory signal.

Current reference strategy

ReferenceBOSRetrace v1.0 is a long-only reference implementation for 15m/1h. It uses a recent bullish BOS/CHoCH, retracement into an open FVG/OB/Fib zone, bullish confirmation with volume, ATR% and EMA proximity, a psychology filter, and staged targets.

This is not the user's personal strategy. It is a code reference. The final system should expose a `StrategySpec/StrategyProfile` interface and wait for the user's actual rules rather than silently treating this reference as the personal edge.

Required final strategy contract

```
StrategySpec
- prerequisites
- setup
- trigger
- confirmations
- invalidation
- entry rule
- stop rule
- TP1/TP2/TP3
- timeframe hierarchy
- regime filters
- liquidity filters
- psychology filters
- derivatives filters
- session rules
- long/short
- risk constraints
- version + provenance
```

10. Psychology Engine

Current implementation

The current engine explicitly labels psychology signals as proxies. It combines:

- crowding proxy from public tape-side imbalance (semantics not fully verified)
- overbought/exhaustion from RSI + volume-z relationships
- external Fear & Greed index (daily, stale after 48h)

Target

Psychology should become a **Market Psychology State** derived from measurable market behavior: crowding, OI changes if TTT exposes them, funding if TTT exposes it, liquidation pressure if TTT exposes it, flow, volatility, displacement and exhaustion. Never claim to literally read human emotions.

11. Derivatives Intelligence - TTT-only strategy

Final data policy

Use only verified TTT public data channels. The current code's derivative adapter demonstrates how a capability registry can be structured, but the final build must not silently substitute Binance or another exchange just because TTT does not expose an endpoint.

Desired capability surface

Metric	Ideal TTT state	If no verified public channel exists
open_interest	Measured/derived from TTT	Unavailable + provenance
funding_rate	Measured/derived from TTT	Unavailable + provenance
mark_price	Measured/derived from TTT	Unavailable + provenance
index_price	Measured/derived from TTT	Unavailable + provenance
basis/premium	Measured/derived from TTT	Unavailable + provenance
long_short_ratio	Measured/derived from TTT	Unavailable + provenance
taker volume	Measured/derived from TTT	Unavailable + provenance
orderbook/depth	Measured/derived from TTT	Unavailable + provenance
CVD	Measured/derived from TTT	Unavailable + provenance
liquidations	Measured/derived from TTT	Unavailable + provenance

UI behavior

Every derivative value must have a visible source/provenance label and a freshness age. If unavailable, do not display 0.0000, N/A hidden, or a green check.

12. AI Architecture and local-AI roadmap

Current AI layer

- provider interface: OpenAI / Anthropic / Gemini / xAI plus deterministic heuristic fallback
- structured JSON verdict
- confidence adjustment
- AI call audit in DB
- optional chart-image input
- annotation validation

Final AI role

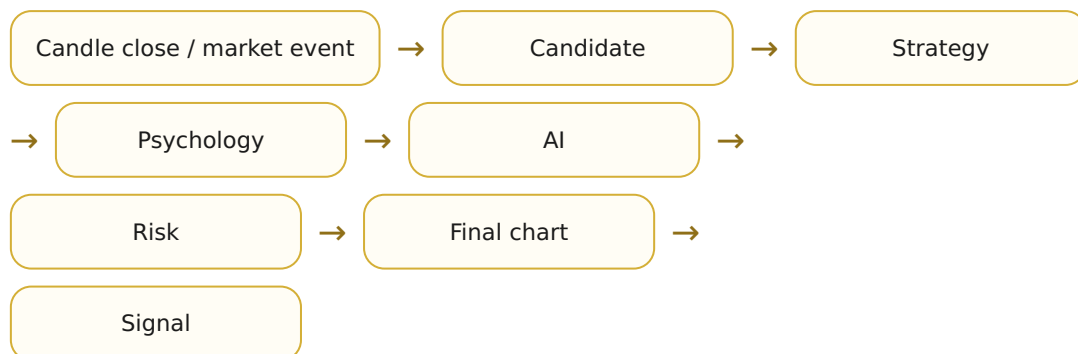
AI should consume: TTT candles + indicators + structure + MTF + psychology + TTT derivatives (when available) + data quality + deterministic chart image. It may confirm, downgrade or reject under policy, but it cannot bypass risk.

Local AI on the user's Victus

The user has reported 32GB RAM, Intel i7-11800H and RTX 3050 Ti 4GB. A realistic local-AI strategy is to use a local runtime such as Ollama on Windows and start with a small quantized coding/reasoning model. Current Ollama docs provide native Windows support and a local API on `localhost:11434`; the current library includes Qwen coder variants, including 7B-class quantizations that can live partly in RAM when VRAM is limited. This is an **optional development-time lane**, not a reason to hard-code AsA to one model.

Official references: Ollama Windows download/docs and library pages. See source list.

13. Opportunity Engine - exact desired semantics



Hard gates

- Data quality gate
- strategy validity gate
- AI policy gate
- risk gate
- dedupe/cooldown gate

Important state distinctions

State	Meaning
NO TRADE	enough valid data; strategy did not yield a valid setup
INSUFFICIENT DATA	analysis could not be performed honestly
RISK BLOCKED	candidate exists but hard risk gate failed
AI REJECTED	candidate exists but policy allows AI to reject
READY	all gates passed and advisory signal is eligible
DELIVERED	signal was actually delivered to configured channels

14. Risk Engine and exit policy

Risk checks already present

- stop distance
- R:R floor
- leverage cap
- liquidation distance
- max positions
- unique symbol
- portfolio exposure
- daily loss
- correlation
- drawdown circuit breaker

Precision

Price rounding must use actual tick size; quantity rounding must use actual step size; minimum notional and leverage tiers must be respected. Precision and increment are not the same thing.

Exit policy

Current staged exit policy is 50% at TP1, then stop to breakeven, with later stages. The system must keep a single source of truth for this policy across live resolver, backtest, chart, Telegram and journal.

No hidden execution. Risk sizing is advisory based on declared equity and market metadata. It is not tied to a real exchange account balance by design.

15. Signal, Telegram and human loop

Current design

The signal engine consumes opportunity events, builds the human message, and uses a durable outbox for channel dispatch. Telegram is dry-run without configured token/chat ID.

Final message content

- symbol + direction + timeframe
- entry / SL / TP1 / TP2 / TP3
- R:R
- risk
- confidence + deterministic breakdown
- strategy
- psychology summary
- relevant TTT derivatives summary when available
- invalidation
- key evidence + contradictions
- final annotated chart
- ADVISORY ONLY - execution is a human decision

Delivery truth

`queued` means not yet delivered. `dispatched` must only mean confirmed delivery for the configured channel(s). Provider message IDs and retry/error state must be retained.

16. Database - persistent memory

Core tables

Table	Purpose
candles	TTT OHLCV + closed/source flags
trades	short tape retention
opportunities	full candidate and lifecycle record
signals	advisory signal record
ai_calls	AI audit record
hypotheses	research hypotheses
kv	configuration/cache values
events	bounded audit/event stream
journal	human execution reconciliation
outbox	durable per-channel delivery queue
jobs	persistent backtest/research job state

Why SQLite now

Single-node development and current universe are suitable for SQLite/WAL. The documented migration seam to Postgres/Timescale exists if the system grows beyond a single node.

Runtime artifact separation

The source package should not ship with a huge runtime DB/WAL/log/charts archive. Keep runtime data external or in a separately managed `data/` directory.

17. API surface - frontend contract

System

`/api/system/health` , `/api/system/status` , `/api/system/events` , `/api/system/config` , `/api/system/config/risk` , `/api/system/config/ai` , `/api/system/logs` .

Market

`/api/market/symbols` , `/api/market/prices` , `/api/market/candles` , `/api/market/trades` , `/api/market/derivatives` , `/api/market/deriv_features` , `/api/market/board` , `/api/market/coverage` .

Analysis + AI

`/api/analysis/{symbol}/{tf}` , `/api/ai/analyze` .

Research

`/api/research/hypotheses` , `/api/research/strategies` , `/api/research/ai-calls` , `/api/research/backtest` , `/api/research/backtest/{job_id}` .

Signals

`/api/opportunities` , `/api/signals` , `/api/signals/{sig_id}` , journal routes, `/api/charts/{fname}` .

Contract rule

The source docs currently show generic OpenAPI response schemas such as `string` in multiple places even when runtime returns objects. A polished frontend should consume typed payloads via a centralized client adapter and the backend should eventually publish accurate Pydantic response models.

18. WebSocket and real-time UI behavior

Current protocol

```
Server -> client
hello
status
price
candle
opportunity
signal
resolved

Client -> server
{"type": "prices", "symbols": ["BTCUSD", "ETHUSD"]}
```

Frontend rules

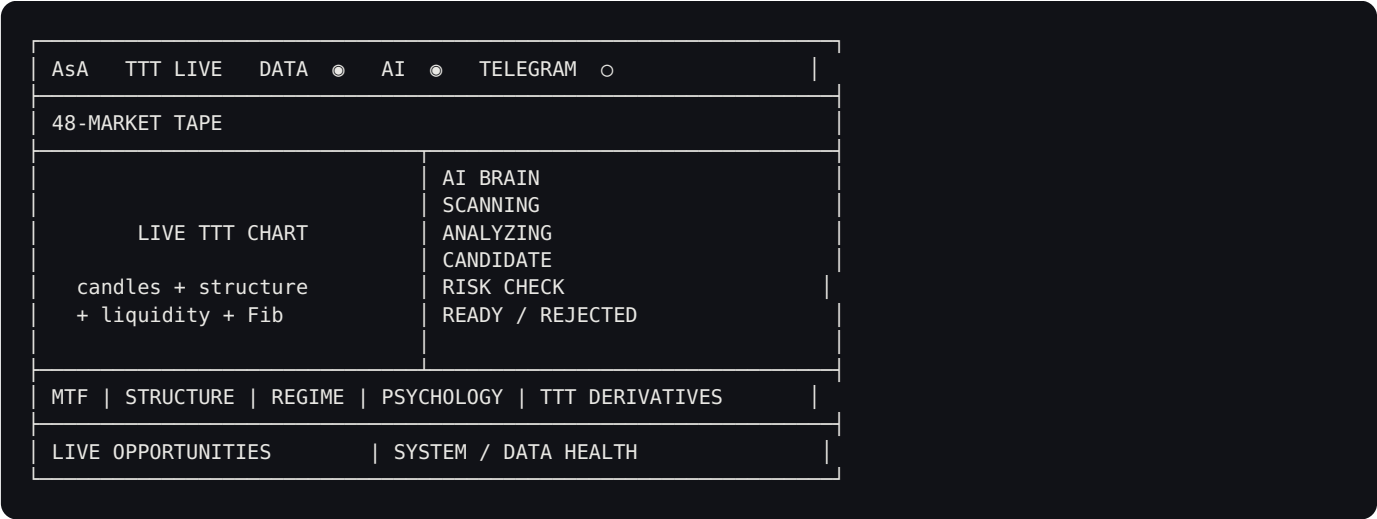
- Show a live connection indicator that is driven by actual WS state.
- When WS drops, fall back to REST and show DEGRADED rather than freezing the UI silently.
- Do not animate price changes that did not arrive from the backend.
- Keep source/age visible on price and chart metadata.

19. Frontend product architecture - what Fable should build

Navigation

Page	Primary purpose
Dashboard	AI command center
Market	48-symbol watchlist
Chart	full trading chart + intelligence
Signals	signal feed + detail
AI	AI audit/analysis
Research	research/backtest outputs
System	runtime/data health
Settings	configuration with masked secrets

Dashboard composition



20. Visual design system

Palette

Token	Intent
Obsidian #0A0B0E	main background
Panel #111217	cards / panels
Champagne Gold #D4AF37	brand/accent
Text #ECEAE4	primary text
Muted #8F8F8F	secondary
Green	actual positive state only
Red	actual negative/error state only

Rules

- Gold is an accent, not wallpaper.
- Do not make every box a card; use hierarchy.
- Chart should occupy the largest area on desktop.
- Use micro-motion for live state, not decorative fireworks.
- Avoid excessive neon, gradient overload, giant empty panels and crypto-cliché visuals.

21. Browser truth states and empty-state UX

State	Visual	Text example
LIVE	green dot + timestamp	TTT LIVE · 83 ms · updated 0.8s ago
STALE	amber dot	TTT STALE · last update 18.4s ago
THROTTLED	amber + throttle icon	TTT THROTTLED · retrying with backoff
INSUFFICIENT DATA	neutral/amber	INSUFFICIENT DATA · 14 closed candles
UNAVAILABLE	muted	Open interest unavailable from verified TTT public feed
AI OFFLINE	muted/amber	AI provider unavailable · heuristic floor active
TELEGRAM DRY-RUN	muted	Telegram not configured · advisory message stored locally

Never use an empty zero. A blank market table, zero funding, zero OI, or zero opportunities must not be used as a fake proxy for unavailable data.

22. Existing research project - what is available

Earlier research system

The older research project is an autonomous crypto strategy research laboratory with 140 catalogued strategy ideas, 102 formalized deterministic rules and 61 backtested ideas across 271 days, 48 symbols and 15m/1h/4h. It explicitly excluded DTM from scope.

Research conclusion

The research report identifies one defensible robust family: a **regime-gated volatility breakout at 4H**. It is described as robust but **not a Champion** because multiple-testing correction and sample limitations prevent a Champion claim.

Key report figures include a median verified-cost expectancy around **+0.165 R** for the leading family and median OOS expectancy around **+0.086 R**, with walk-forward positivity in 3/5 folds. These are research outputs, not promises or guarantees.

Do not collapse research and live product into one label. A strategy can be registered as experimental/validated/rejected; live opportunity generation must still respect current data quality and live risk gates.

23. Research registries and provenance model

The earlier research project contains registries for strategies, sources, claims, ideas, traders, videos, bugs, experiments, champions and rejected results. The final AsA should integrate these through a small adapter/import layer rather than copying the entire historical repository into the runtime app.

Suggested provenance fields

```
id
version
source_type
source_ref
claim_or_hypothesis
data_window
universe
timeframes
validation_status
metrics
notes
last_verified_ts
```

24. Backtest and validation integrity

Current engine rules

- signal on closed bar i
- fill at next bar open
- conservative SL-before-TP when same-bar path is unknowable
- fees + slippage per fill
- staged exits
- R against original stop

Validation tools

- OOS split
- walk-forward
- Monte Carlo order resampling
- parameter sensitivity
- cost robustness
- regime breakdown
- bootstrap CI
- multiple-testing warning / deflated-performance style gate

Known limitations

Funding is not modeled if the source does not provide historical funding. Partial fills, queue position, spread and exact intrabar path are not fully modeled. The system reports these limitations rather than hiding them.

25. Security and execution boundary

Threat	Current control
Trading credentials	none by design
Order execution	no API route / no execution client
Secrets	env-only + masking + secret scan
DB SQL injection	parameterized SQL
XSS	typed DOM construction; no raw API HTML
Chart path traversal	filename validation under charts dir
Mutation auth	X-API-Token when configured

Never add exchange execution. The entire product identity depends on remaining advisory-only until the human decides and executes separately.

26. Local development and local-AI plan

Recommended workstation role

The user's Windows HP Victus can be the primary development workstation. The local 32GB RAM / i7-11800H / RTX 3050 Ti 4GB profile is sufficient for backend/frontend development, backtests, databases, chart rendering and a modest local inference layer.

Development stages

- Stage 1: run AsA backend locally and verify TTT live data.
- Stage 2: build and visually refine the web terminal.
- Stage 3: integrate local AI service behind a provider-agnostic interface.
- Stage 4: add structured memory and local research retrieval.
- Stage 5: only after the full local system is stable, consider VPS deployment.

Ollama example

```
irm https://ollama.com/install.ps1 | iex  
ollama run qwen2.5-coder:7b-instruct-q4_K_M
```

This example is a local development option, not a required model choice. Verify memory/VRAM behavior on the actual machine before pinning a model.

27. Fable implementation sequence

Phase A - Real shell

- connect to existing backend
- load status
- load prices
- show TTT provenance
- render 48-symbol tape

Phase B - Chart

- load BTC 15m/1h/4h
- build chart container
- wire live candle/price events
- render actual overlays
- add source/age indicators

Phase C - Intelligence

- MTF panel
- structure
- regime
- psychology
- derivatives availability
- AI BRAIN
- opportunity cards

Phase D - Signal detail

- final chart image
- risk
- AI evidence
- Telegram delivery state
- manual journal

Phase E - Hardening

- responsive/mobile
- error states
- offline/degraded states
- contract validation
- browser E2E

- no-mock-data scan

28. Acceptance checklist - product cannot be called done before these pass

#	Acceptance test	Pass condition
1	48 symbols	All 48 configured symbols render from backend
2	TTT provenance	Price/candle UI visibly identifies TTT as source
3	Live age	Age updates and stale state is visible
4	15m/1h/4h	Real candle windows load, not six fake candles
5	Chart fidelity	Chart matches backend candle values
6	Structure	Only backend-confirmed geometry is drawn
7	MTF	4H/1H/15M context is coherent
8	Data gaps	Insufficient history is shown explicitly
9	Derivatives	Only verified TTT values appear; missing metrics stay unavailable
10	AI	Provider status matches backend; no fake online
11	Opportunity	Cards come from real opportunity endpoint/events
12	Risk	Visible numbers match backend risk object
13	Telegram	Connected only after actual configured delivery
14	No other venue	No TradingView/Binance market-data values in UI
15	Responsive	Desktop and mobile are intentionally designed
16	No mock market data	Static source scan finds no hardcoded live prices/candles
17	Browser E2E	Happy-path and degraded-path browser tests pass
18	Performance	UI remains responsive under live event flow

29. Anti-patterns - explicit things Fable must NOT do

Do NOT make a pretty shell and call it AsA.

- Do not embed sample BTC/ETH prices in JavaScript.
- Do not use chart images from TradingView or other websites as fake live charts.
- Do not use Binance/other-exchange data to fill TTT gaps in the final terminal.
- Do not show "AI ONLINE" if no provider is configured.
- Do not show "Telegram Connected" if no delivery occurred.
- Do not show a zero when the value is unknown.
- Do not build separate, duplicated trading logic in frontend JavaScript.
- Do not make the whole interface a grid of cards.
- Do not create dozens of files for a visual redesign when a small coherent structure is enough.

30. Source-derived current documentation notes

Current project claims that are important to preserve

- Current README says AsA is intelligence/research, never execution, with TTT as primary and Binance failover.
- Current docs record TTT REST endpoints and the lack of a verified TTT WebSocket in the adapter.
- Current status report records live TTT prices for 48 symbols and a 66-pass/1-skip test claim from the project environment.
- Current limitations document records TTT throttling, candle lag, no native 1d, no verified funding/OI/liquidation/L2 in that build, and explicit unavailable behavior.
- Current runtime testing supplied by the user showed real TTT prices for 48 symbols and real Binance derivative values; those cross-venue values are precisely why the final TTT-only UI must be provenance-strict.

Current runtime observation from user testing

The observed runtime showed TTT source prices for the 48-symbol set, while the derivative endpoint returned measured values from Binance for funding, mark/index, long/short, taker volume and orderbook. The user explicitly rejected that architecture for the final product. Therefore this document treats the TTT-only rule as a **product-level override**.

31. Fable 5.1 build brief - condensed version

This page is a compact reminder; the full copy-paste prompt is included at the end as plain text.

You are Fable 5.1 acting as the Senior Product Designer + Frontend Architect + UI Engineer for AsA. You are NOT being asked to invent a generic crypto dashboard. You are being asked to build the FRONTEND EXPERIENCE for an existing AI-native crypto futures trading intelligence system.

READ THE ATTACHED DOCUMENT "AsA Master Build Bible v1.0" completely before coding. Treat the document as the product/technical specification and source-of-truth summary.

HARD RULES

1. TTT is the ONLY market-venue source for AsA.
2. Do NOT use TradingView, Binance, Coinbase, CoinMarketCap, CoinGecko, or any other exchange/site as a market-data source.
3. The existing backend may expose cross-provider capability code, but the final UI/product must treat TTT as the primary and only venue truth. If a metric is not available from TTT, show UNAVAILABLE rather than substituting another venue.
4. NEVER fabricate prices, candles, OI, funding, volume, order book, signals, AI status, Telegram status, or trade results.
5. NEVER embed demo/mock market numbers in production UI.
6. When backend data is absent, display a precise state: CONNECTING / STALE / UNAVAILABLE / INSUFFICIENT DATA.
7. AsA is ADVISORY ONLY. There is no trade execution.
8. The human is always the final execution decision.
9. The chart must be based on real TTT candle/market data supplied by the backend.
10. The UI is the product, not a decorative skin around APIs.

GOAL

Build a premium black/obsidian + restrained champagne-gold AI Trading Intelligence Terminal that makes the user feel they are operating a living market-analysis system. The experience must be chart-first, live, honest, low-noise, fast, and visually exceptional.

TECHNICAL CONTEXT

- Existing backend is FastAPI/uvicorn.
- Existing API base path is /api.
- Existing WebSocket is /ws.
- Existing frontend stack is plain ES modules + Lightweight Charts.
- Existing backend exposes market, candles, trades, analysis, derivatives/capability, coverage, opportunities, signals, AI, research, system status, logs, settings, and WebSocket feeds.
- Keep the API contract. Build an excellent client for it.
- Do not rewrite the backend unless a tiny adapter is necessary for a clear contract mismatch.
- If you need a backend change, explain it first in a compact TODO list.

PRIMARY UX INFORMATION HIERARCHY

1. Live market state
2. Selected market chart
3. AI analysis state
4. Strategy/psychology/structure evidence
5. Opportunity and risk
6. Signal history
7. System health

DASHBOARD

Create a dense but elegant command center:

- Header: AsA logo/wordmark, TTT LIVE indicator, data freshness, AI state, Telegram state.
- Market strip for the 48-symbol universe.
- Central large live candlestick chart.
- Right-side AI BRAIN stepper: SCANNING -> ANALYZING -> CANDIDATE -> RISK CHECK -> READY / REJECTED / COOLDOWN.
- Below chart: MTF context (4H / 1H / 15M), structure, regime, psychology, derivatives availability.
- Opportunities list with confidence score, entry/SL/TP, RR, risk state.
- System/data health rail.

MARKET PAGE

Show the 48-symbol universe from backend only.

For each row:

- symbol
- live price
- source = TTT
- age/freshness
- 1h change if available
- 24h change if available
- volume if available
- regime
- signal/strategy state
- coverage state
- no fake zeros

CHART PAGE

This is the hero screen.

Support:

- 4H, 1H, 15M as primary analysis hierarchy
- additional backend-supported timeframes
- live price updates
- closed-candle updates
- pan/zoom
- crosshair
- volume
- EMA
- S/R
- swing points
- BOS
- CHoCH/MSS
- FVG
- Order Blocks
- breakers
- liquidity markers
- Fibonacci
- trendlines
- entry/SL/TP1/TP2/TP3
- AI annotations
- source/provenance labels

Every annotation must show on hover/click:

- source: deterministic / AI
- confidence
- concise reason

Do not use fake chart lines.

Every geometry must originate from backend data or a validated AI annotation.

AI PANEL

Do not expose hidden chain-of-thought.

Show only concise evidence:

- Structure aligned
- MTF aligned
- Liquidity sweep detected
- Psychology signal
- Derivatives signal
- Risk passed
- Contradictions
- Invalidation
- Model/provider
- latency
- whether an image was used

AI must never appear "online" just because a UI flag exists.

Render the exact provider status delivered by the API.

OPPORTUNITY DETAIL

When a setup exists, open an institutional-style trade sheet:

- symbol / direction / timeframe
- final annotated chart
- entry / SL / TP1 / TP2 / TP3

- RR
- risk amount
- leverage cap
- liquidation estimate if available
- regime
- MTF
- psychology
- derivatives context if available from TTT
- AI verdict
- thesis
- contradictions
- invalidation
- signal delivery state
- ADVISORY ONLY

SIGNAL DETAIL

- final chart image
- exact signal message
- Telegram status
- opportunity status
- lifecycle stage
- manual execution journal

SYSTEM PAGE

Expose truth, not decoration:

- TTT connection
- last data update
- per-symbol freshness
- candle coverage by TF
- backfill progress
- provider errors
- worker states
- queue depth
- AI provider status
- Telegram delivery status
- DB size
- latency metrics

SETTINGS

- risk profile
- AI policy
- notification settings
- theme/system preferences
- source status
- all secrets masked
- no fake connected states

VISUAL DESIGN

- Obsidian black background
- restrained champagne-gold accent
- green/red only for actual market direction/state
- subtle glass surfaces
- thin borders
- strong typography
- no excessive gradients
- no dashboard-card explosion
- no giant empty hero areas
- no childish neon crypto style
- micro-animations only for state transitions/live updates

RESPONSIVE

Desktop: chart-centric terminal.

Laptop: preserve chart height and density.

Mobile: dedicated composition, not a squeezed desktop. Use bottom navigation and collapsible intelligence panels.

REAL-TIME BEHAVIOR

Use WebSocket for status/price/candle/opportunity/signal events when connected.

REST polling is the fallback for UI refresh.

Show actual connection status.

Do not simulate ticks.

Price UI must show source=TTT and data age.

ERROR STATES

Design explicit states for:

- TTT disconnected
- TTT throttled
- stale price
- stale candle
- insufficient history
- unavailable metric
- empty opportunity list
- AI unavailable
- Telegram not configured
- API unauthorized
- backfill in progress

CRITICAL ACCEPTANCE TESTS

1. Open dashboard with real backend.
2. See 48 symbols.
3. Confirm displayed price source is TTT.
4. Open BTCUSDT.
5. Load 15M, 1H, 4H.
6. Verify the candles are real backend candles, not fabricated.
7. Verify live age changes.
8. Verify chart annotations correspond to API data.
9. Verify a missing metric renders unavailable, never zero/fake.
10. Verify AI state matches API provider state.
11. Verify no screen claims Telegram is connected unless backend says so.
12. Verify no screen contains TradingView/Binance market data.
13. Verify responsive mobile composition.
14. Verify no mock market numbers exist in source.

DELIVERABLE

Create the frontend so the user can run the backend and immediately open a real AsA terminal.
The first impression must be: "this is an AI trading intelligence system that is actually alive."
Do not answer with a concept. Build the UI.
Do not hide incomplete backend capabilities. Expose them honestly.
Do not replace AsA with a generic dashboard template.

32. Exact frontend file targets

Current frontend files

Path	Role
frontend/css/style.css	existing frontend/vendor asset
frontend/index.html	existing frontend/vendor asset
frontend/js/api.js	existing frontend/vendor asset
frontend/js/main.js	existing frontend/vendor asset
frontend/js/pages/ai.js	existing frontend/vendor asset
frontend/js/pages/chart.js	existing frontend/vendor asset
frontend/js/pages/dashboard.js	existing frontend/vendor asset
frontend/js/pages/logs.js	existing frontend/vendor asset
frontend/js/pages/market.js	existing frontend/vendor asset
frontend/js/pages/research.js	existing frontend/vendor asset
frontend/js/pages/settings.js	existing frontend/vendor asset
frontend/js/pages/signals.js	existing frontend/vendor asset
frontend/js/pages/system.js	existing frontend/vendor asset
frontend/js/util.js	existing frontend/vendor asset
frontend/js/ws.js	existing frontend/vendor asset
frontend/vendor/lightweight-charts.standalone.production.js	existing frontend/vendor asset

Fable should prefer a small, coherent module structure. A full UI rebuild does not justify bundling a framework or adding a build system unless the user explicitly wants one.

33. Test inventory

Test file	Scope
conftest.py	automated test / fixture
e2e_browser.js	automated test / fixture
binance_exchangeinfo.json	automated test / fixture
ttt_candles.json	automated test / fixture
ttt_markets.json	automated test / fixture
seed_e2e_fixture.py	automated test / fixture
test_ai_structured.py	automated test / fixture
test_ai_veto.py	automated test / fixture
test_annotations.py	automated test / fixture
test_api.py	automated test / fixture
test_backtest.py	automated test / fixture
test_binance_filters.py	automated test / fixture
test_confidence.py	automated test / fixture
test_contracts.py	automated test / fixture
test_derivatives.py	automated test / fixture
test_exit_policy.py	automated test / fixture
test_latency.py	automated test / fixture
test_meta.py	automated test / fixture
test_normalize.py	automated test / fixture
test_opportunity.py	automated test / fixture
test_psychology2.py	automated test / fixture
test_regime.py	automated test / fixture
test_restart_survival.py	automated test / fixture
test_risk.py	automated test / fixture
test_security.py	automated test / fixture
test_signal_message.py	automated test / fixture
test_strategy.py	automated test / fixture

Test file	Scope
test_structure.py	automated test / fixture
test_trendlines.py	automated test / fixture
test_ttt_parser.py	automated test / fixture

The existing test suite covers normalization, TTT parser, structure, regime, strategy, risk, opportunity, AI structured output/veto, annotations, contracts, derivatives, exit policy, latency, metadata, API, restart survival, security, signal message and trendlines. Browser E2E is represented by `tests/e2e_browser.js`, but actual GUI execution should still be confirmed on the Windows workstation.

34. Research findings worth preserving

Earlier research scale

Item	Reported value
Research duration	271 days
Universe	48 symbols
Core TFs	15m / 1h / 4h
Ideas catalogued	140
Formalized	102
Backtested	61
Strategies recorded in final registry	68
Sources	51
Traders/authors	10
Bugs registry	34

Leading robust family

Regime-gated volatility breakout, 4H. The research report calls it robust but explicitly reports **CHAMPIONS = 0** due multiple testing and sample limitations. This should become a research artifact that can be imported into AsA, not a hardcoded live strategy.

35. References and provenance

Project sources

- AsA current workspace: README.md, docs/ARCHITECTURE.md, docs/API_SPEC.md, docs/DATABASE.md, docs/LIMITATIONS.md, docs/SECURITY.md, docs/STATUS_REPORT.md, docs/ADR.md.
- AsA source tree under backend/asa, frontend, tests.
- Previous FINAL AI TRADE research project: README.md, FINAL_AI_TRADE_STRATEGY_RESEARCH_REPORT.txt, registry files and research/backtest results.
- Initial TTT-based project: Final_AI_Trade_v0.1.2_NO_TON.zip and README_FA.md.

Current external TTT references

- TTT official home / product pages: <https://www.thetruetrade.io/>
- TTT perpetual trading mechanics and fee/funding help article (published Aug 29, 2026): <https://www.thetruetrade.io/help-center/article/46674374193425>
- TTT futures contract pages expose current contract/market concepts such as OI, funding, mark/index and risk-limit fields in the product UI.
- Official TTT pages also expose trading pages with chart, order book and recent trades UI.
- Official Ollama Windows and model-library references were used only for the optional local-AI roadmap.

External web references were checked in the current date window. They are included to distinguish what TTT publicly presents in its product from what is actually verified as a public API endpoint in the codebase.